# Fetch API Daily Coding Assignment

## 1. Introduction

This assignment involves developing a frontend dashboard that displays blog posts and a todo list retrieved from remote servers using the Fetch API. The application demonstrates asynchronous data handling, DOM manipulation, proper error handling, and structured JavaScript design using a suitable design pattern.

## 2. Objective

The objective of this assignment is to:

- Retrieve blog posts and todo data from public APIs.
- Dynamically display the data on a web page.
- Implement proper error handling.
- Refactor the code using a JavaScript design pattern.
- Build a clean and responsive UI.

## 3. Tools & Technologies Used

- HTML5
- CSS3
- JavaScript (ES6)
- Fetch API
- JSONPlaceholder Public API

## 4. Implementation Details

### 4.1 Fetch API Implementation

The application retrieves data from:

- https://jsonplaceholder.typicode.com/posts
- https://jsonplaceholder.typicode.com/todos

The fetch() function is used with async/await for asynchronous execution.

Example:

```
async function fetchData(url) {
const response = await fetch(url);
if (!response.ok) {
throw new Error("Server error");
}
return await response.json();
```

```
}
```

### 4.2 DOM Manipulation

- Elements are dynamically created using document.createElement().
- Data is inserted using appendChild().
- Existing content is cleared before re-rendering to avoid duplication.

### 4.3 Error Handling

Proper error handling was implemented using:

- try...catch
- Response validation using response.ok
- Displaying user-friendly error messages
- Independent error sections for posts and todos

### 4.4 JavaScript Design Pattern

The application uses the **Revealing Module Pattern** to maintain clean structure and prevent global scope pollution.

Example:

```
const App = (function () {
function init() {
// initialization logic
}
return { init };
})();
```

Benefits:

- Encapsulation
- Better organization
- Improved maintainability
- Scalable structure

## 5. UI Enhancements

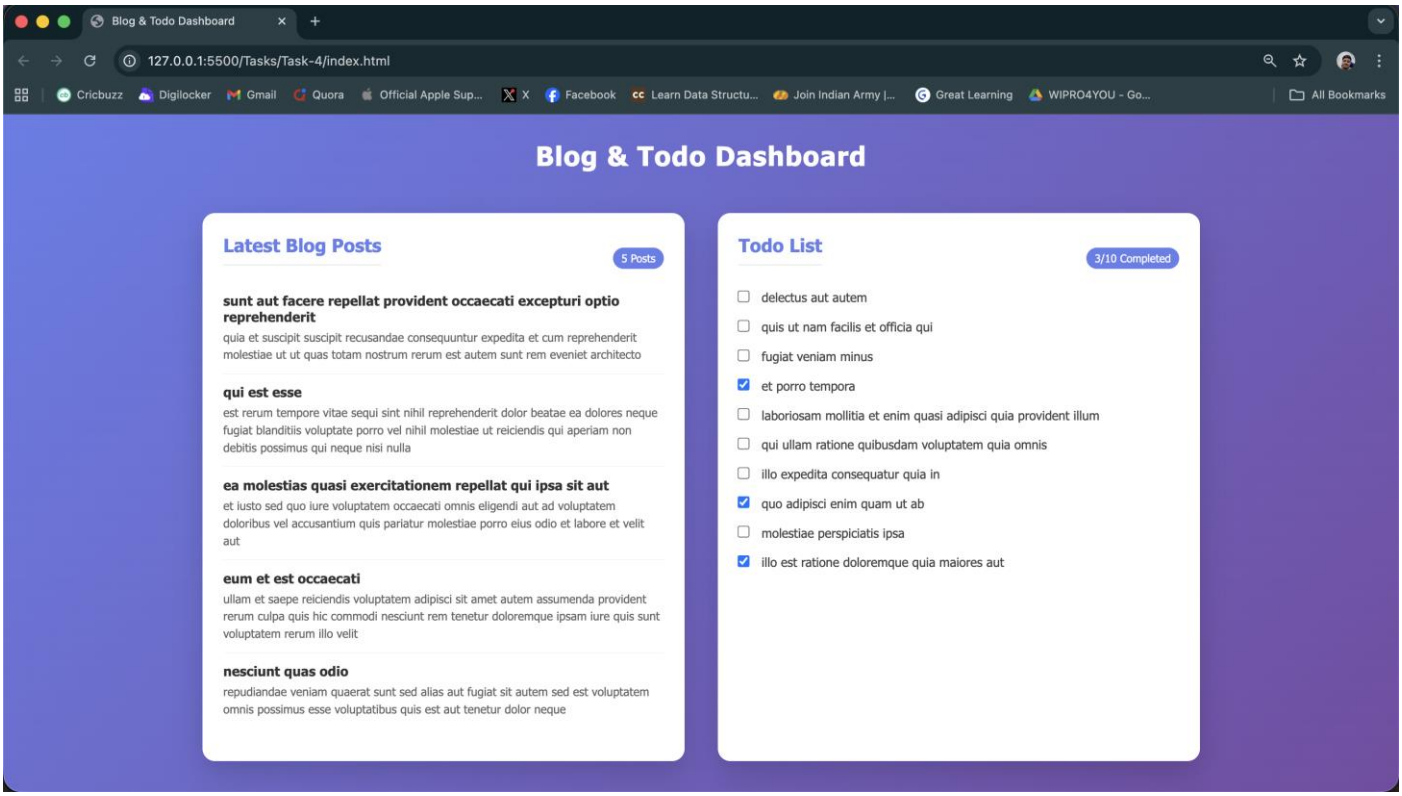
The application includes:

- Responsive card-based layout
- Loading indicators
- Post count badge
- Completed todo statistics
- Hover effects and clean design

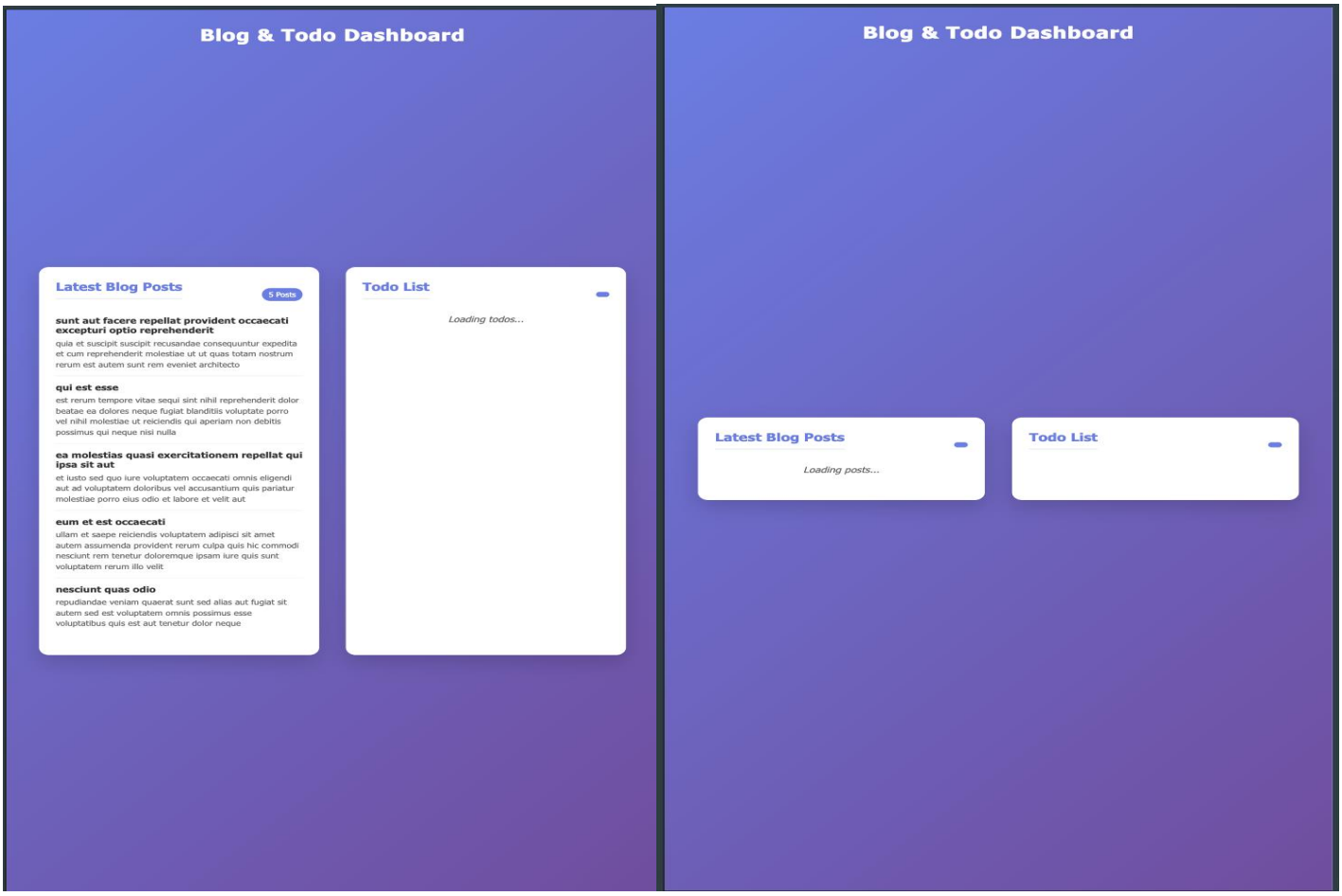
# 6. Challenges Faced

- Managing asynchronous API calls properly.
- Ensuring UI stability during errors.
- Structuring JavaScript using a design pattern.
- Designing a clean and responsive interface.

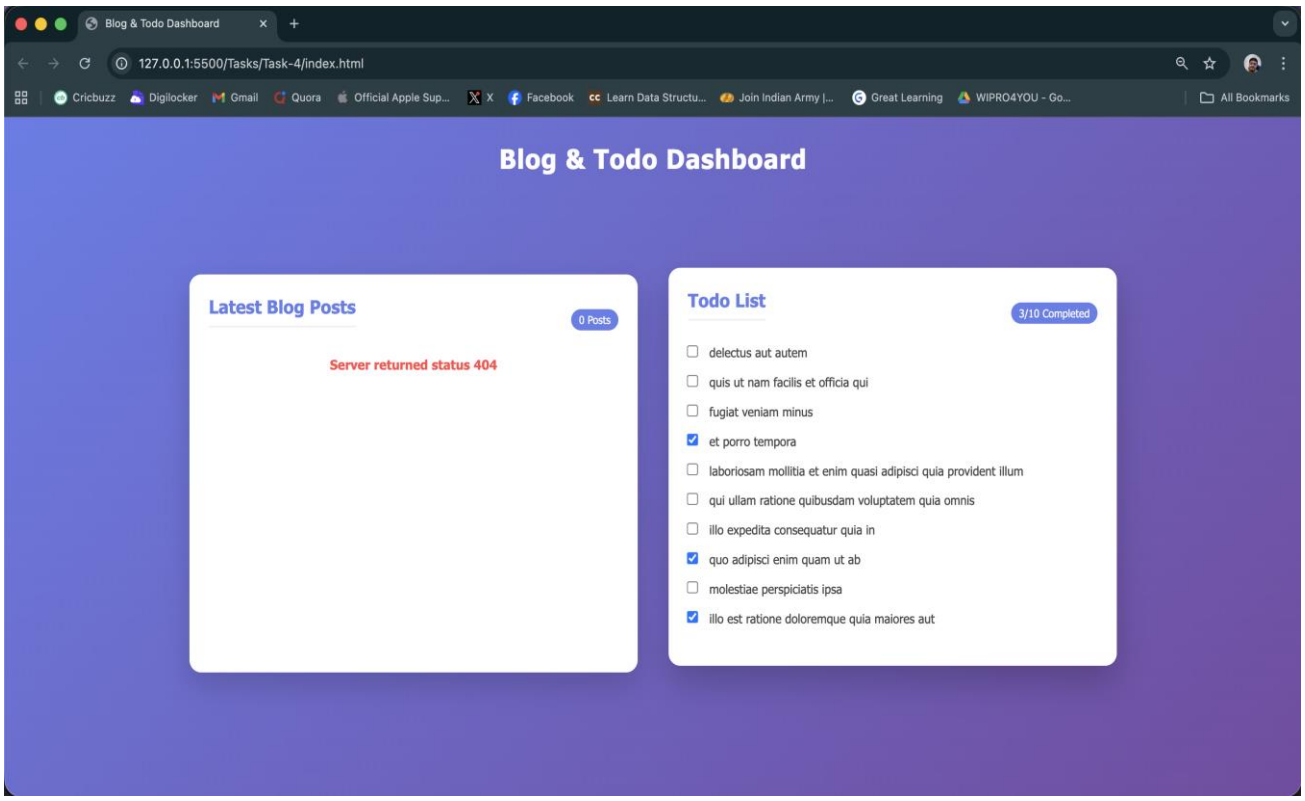
# 7. Output Screenshots




**Normal working output (Posts and Todos displayed)**




**Loading state**




**Error handling demonstration**

## 8. Reflection

This assignment improved my understanding of:

- Fetch API and asynchronous JavaScript
- Working with REST APIs
- Handling JSON data dynamically
- DOM manipulation
- Implementing structured error handling
- Writing modular JavaScript using design patterns

The most important learning outcome was understanding how structured code organization improves readability, scalability, and maintainability.

## 9. Conclusion

The application successfully fulfills all assignment requirements:

- Fetches and displays blog posts and todos
- Implements proper error handling
- Uses a JavaScript design pattern
- Provides a clean and responsive UI